

Chapter 4: Milvus Vector Database Architecture & Operations

Complete Technical Guide & Code Inventory for 04-VectorDB/Milvus/ (20 Code Files & Subfolders)

Milvus is an open-source, highly scalable vector database designed for high-performance similarity search and AI applications. The 04-VectorDB/Milvus/ directory provides a comprehensive, hands-on engineering codebase covering database orchestration, schema definition, entity CRUD, vector index algorithm selection, metric types, advanced scalar-filtered search, full-text search, and pagination.

1. Root Files & Infrastructure Setup

Filename	Type	Core Function & Execution Mechanism
docker-compose.yml	Docker Config	Spins up local Milvus Standalone environment with 3 containers: Milvus engine, Etcd (metadata store), and MinIO (object storage for vectors/log files).
create_milvus_db.py	Setup Script	Initializes Milvus database connections via connections.connect() and creates isolated logical databases.
a-working-sample.py	End-to-End Demo	Complete working pipeline: connects to Milvus, creates collection schema, generates dummy vectors, builds HNSW index, loads collection, and executes similarity search.
.env.example	Environment Config	Specifies host URI (localhost:19530 or Milvus Lite ./milvus_demo.db), port, credentials, and default collection names.

2. Subfolder: 01-CollectionsAndEntities

Focuses on multi-tenancy, collection lifecycle, strongly-typed schema creation, dynamic fields, and entity CRUD operations.

Filename	Primary Concepts	Technical Execution & Milvus API Usage
01-database.py	Logical DB Isolation	Uses db.create_database() and db.using_database() for multi-tenant data isolation within a single cluster.
02-collection.py	Collection Lifecycle	Creates collections via Collection(), handles collection.load() into RAM for search, and collection.release() to free memory.
03-schema.py	Schema & Field Types	Defines structured fields with FieldSchema (INT64 primary key, VARCHAR text, FLOAT_VECTOR, enable_dynamic_field=True for flexible JSON attributes).
04-entity(data).py	Entity CRUD	Demonstrates collection.insert() for bulk data loading, collection.upsert() for entity updates, and deletion via expression collection.delete(expr='id in [1, 2]').

3. Subfolder: 02-Indexes (Vector Index Algorithms)

Examines vector index structures, trading off search speed, memory footprint, build time, and recall precision.

Index File	Index Type	Key Parameters	Recall vs Latency & Memory Profile
------------	------------	----------------	------------------------------------

01-milvus_flat_index.py	FLAT	index_type: FLAT	100% Recall (Brute Force). High search latency, zero quantization compression. Best for small datasets (<10k vectors).
02-ivf_flat_index.py	IVF_FLAT	nlist: 1024 nprobe: 16	Clusters space into nlist Voronoi cells. Searches only nprobe nearest cells. Fast speed, high recall (~95%), moderate RAM.
03-ivf_pq_index.py	IVF_PQ	m: 8 nbits: 8	Product Quantization. Compresses vectors into m byte codes. Low RAM usage , ultra-fast search, slight recall loss.
04-hnsw_index.py	HNSW	M: 16, efConstruction: 200 ef: 64	Graph-based index. Ultra-high QPS and Recall (>98%). Requires high RAM to maintain multi-layer graph structures.
05-DiskANN.py	DISKANN	search_list: 32	Disk-resident graph index. Holds vectors on NVMe SSD, keeping only structural graph in RAM. Enables billion-scale search on budget hardware.

4. Subfolder: 03-SearchAndMetrics

Covers query execution modes, distance metrics, metadata pre-filtering, range bounds, group deduplication, and full-text search.

Filename	Search Feature	Key Parameters & Code Syntax
01-basic-ann.py	Basic Similarity Search	collection.search(data=[query_vec], anns_field='vector', param={'metric_type': 'L2', 'params': {'nprobe': 10}}, limit=5)
02-ann-diff-metrics.py	Distance Metrics	Compares L2 (Euclidean distance), IP (Inner Product), and COSINE distance metrics for normalized vectors.
03-filtered-search.py	Scalar Pre-Filtering	Attaches boolean expression string: expr='age > 25 and category == "tech"' before vector similarity matching.
04-range-search.py	Similarity Range Query	Filters matches by distance bounds using param={'radius': 0.8, 'range_filter': 0.5} (returns vectors within threshold range).
05-group-search.py	Group-by Deduplication	Deduplicates results by metadata field using group_by_field='doc_id' (returns top match per document).
06-full-text-search-bm25-ch.py	Chinese Sparse BM25	Uses Milvus native Chinese tokenizer & sparse BM25 generator (sparse_float_vector) for full-text search.
06-full-text-search-bm25-en.py	English Sparse BM25	Uses Milvus native English analyzer & BM25 function to index and search sparse lexical keywords.
07-text-match.py	Text Substring Match	Executes exact keyword and wildcard text matching using expr='TEXT_MATCH(title, "Wukong")'.
08-search-iter.py	Search Pagination	Iterates over thousands of vector results efficiently using collection.search_iterator(batch_size=100).
09-metadata-query.py	Pure Attribute Query	Fetches records matching scalar logic without vector calculations using collection.query(expr='id in [1, 2, 3]').

5. Index & Metric Selection Guide for Production

Scenario / Requirement	Recommended Index	Metric Type	Why Choose This Configuration?
------------------------	-------------------	-------------	--------------------------------

Production Standard RAG (High QPS)	HNSW	COSINE / IP	Delivers ultra-high queries per second and >98% recall. Best for enterprise memory-backed RAG.
Billion-Scale Vectors (RAM Constrained)	DISKANN / IVF_PQ	L2 / COSINE	DISKANN offloads vector data to NVMe SSD; IVF_PQ compresses vectors by 8x–16x to fit in limited RAM.
Filtered E-Commerce / Doc Search	HNSW + Scalar Expr	IP / COSINE	Pre-filters scalar metadata attributes (e.g. category, price, date) before vector similarity calculation.
Lexical Keyword + Semantic RAG	BM25 Sparse + HNSW	IP (Sparse) + COSINE	Native Milvus sparse BM25 handles exact terminology/product IDs while dense HNSW handles semantic intent.